

Reviewer Pack — Symmetric Power Rank Gap in Permanent vs Determinant Polynom...

Entry d27b04e7091e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 05:35:31 UTC

Symmetric Power Rank Gap in Permanent vs Determinant Polynomials

Entry ID: d27b04e7091e Verdict: INCONCLUSIVE Recorded: 2026-05-09 05:35:31 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Symmetric Power Rank of Homogeneous Polynomials

Statement:

For all $n \geq 2$, the symmetric power rank of the permanent polynomial Perm_n is at least $2^{\lceil n/2 \rceil}$ times the symmetric power rank of the determinant polynomial Det_n for exponents $k = \lceil n/2 \rceil$.

Rationale (proposer’s reasoning):

Symmetric power ranks capture the complexity of decomposing polynomials into symmetric tensors, which mirrors the algebraic structure of permanent/determinant. The exponential gap conjectured here would imply inherent structural differences in their representation-theoretic decompositions, potentially separating their complexity classes.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dfa9735e1a8b3c42

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetric power rank permanent determinant representation theory homogeneous polynomials - symmetric power rank exponent permanent determinant homogeneous polynomials - symmetric power rank gap permanent determinant homogeneous polynomials representation theory

Top relevant hits considered: - [http://arxiv.org/abs/1503.00822v2] Product Ranks of the 3×3 Determinant and Permanent - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/1603.02493v4] Clifford theory for glider representations - [http://arxiv.org/abs/1604.00993v1] The 'Core' of Symmetric Homogeneous Polynomial Inequalities of Degree Four of Three Real Variables - [http://arxiv.org/abs/2206.05177v2] m -Symmetric functions, non-symmetric Macdonald polynomials and positivity conjectures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for i in range(1, n+1):
            clause = [random.choice([f'x{i}', f'~x{i}']) for _ in range(3)]
            clauses.append(clause)
        return clauses

    def variable_substitution(clauses, n):
        variables = {}
        for i in range(1, n+1):
            a = random.randint(1, 2)
            b = random.randint(1, 2)
            variables[f'x{i}'] = f'a{a}b{b}'
            variables[f'~x{i}'] = f'a{3-a}b{3-b}'
        return [[variables[var] for var in clause] for clause in clauses]

    def polynomial_from_3cnf(clauses):
        terms = []
        for clause in clauses:
            term = ' + '.join([f'{random.choice(["+", "-"])} {var}' for var in clause])
            terms.append(term)
```

```

        return f'({" * ").join(terms))'

def symmetric_power_rank(poly, k):
    if k == 0:
        return 1
    rank = 0
    n = len(poly.split(' + '))
    for i in range(n):
        subpoly = poly.replace(f'x{i+1}', f'y{i}')
        rank += symmetric_power_rank(subpoly, k-1)
    return rank

n = random.randint(5, 40)
clauses = generate_3cnf(n)
substituted_clauses = variable_substitution(clauses, n)
permanent_poly = polynomial_from_3cnf(substituted_clauses)

det_poly = ' * '.join([f'x{i}' for i in range(1, n+1)])

k = math.ceil(n / 2)
perm_rank = symmetric_power_rank(permanent_poly, k)
det_rank = symmetric_power_rank(det_poly, k)

ratio = Fraction(perm_rank, det_rank)
threshold = Fraction(2**(n/2), 10)

conjecture_holds = ratio >= threshold
counterexample = "" if conjecture_holds else f"Ratio {ratio} < {threshold}"

return {
    "metric_name": "Symmetric Power Rank Ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed=NA")
    else:

```

```
print(f"RESULT: INCONCLUSIVE insufficient evidence to support or falsify the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ba6ff81.py", line 85, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ba6ff81.py", line 66, in run_trial
    threshold = Fraction(2**(n/2), 10)
                  ^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid Fraction arguments; no data produced to assess conjecture.
| next: Fix the Fraction constructor to use integer exponents (e.g., $2^{(n/2)}$) and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82459
2	preregistration	ollama_remote	qwen3:8b	0	0	24374
3	novelty	ollama_remote	qwen3:8b	0	0	20481
4	novelty	ollama_remote	qwen3:8b	0	0	24428
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13191
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9631
7	judge	ollama_remote	qwen3:8b	0	0	22293

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 196858 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/d27b04e7091e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d27b04e7091e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d27b04e7091e.tar.gz` (if generated)